

COMSATS UNIVERSITY ISLAMABAD,
LAHORE CAMPUS



NAME	MINAHIL SEHAR
REG NO.	SP26-BAI-032
COURSE INSTRUCTOR	M SHAHID BHATTI

DATE : 25TH SEPTEMBER

2026

TASK # 1-- Stakeholder and requirement analysis

Stakeholders:

- 1- Students – expect fair access.
- 2- Faculty members – need priority for scheduled teaching and need priority for scheduled teaching.
- 3- Administrators – want high room utilization.
- 4- Maintenance staff – wants to have the ability to block unsafe rooms immediately.

Conflicts and assumptions:

- 1- Students require fair access while faculty members need priority for scheduled and urgent academic activities.
- 2- Administration wants high room utilization while maintenance staff wants to be able to block unsafe room immediately.
- 3- Administration wants accessible rooms fully utilized like other rooms, but if a student who doesn't need accessible features books it, a students who actually needs it may be left without a room.

Facts:

- 1- For scheduled teaching and urgent academic activities faculty members are prioritized, while for non teaching use both compete equally.
- 2- Maintenance staff is prioritized as life safety is more essential than room utilization.
- 3- Students who need accessible features can flag this at booking time. The system uses it to prioritize their request, but only the system disability staff can see the flag.

TASK # 2: Object oriented conceptual mode

Class	Responsibilities	Attributes/Operations
Person	Anyone who uses the system	name, contactInfo, login()
Student	Books room as a learner	studentId, accessibilityFlag (T/F), requestSpace()
FacultyMember	Books space a teaching staff	facultyId, Department, scheduleClass(), prioritizespace()

AdminStaff	Manages overall space utilization and approves requests	adminId, approveReservation(), seeUtilizationReport()
SecurityStaff	Needs traceable access records for monitoring	securityId, viewAccessLog()
Room	A reserveable physical space	roomNumber, roomCapacity, accessibilityStatus()
AccessRequest	Keeps accessibility needs private.	requestId, viewEligibility(), submitRequest()
Notification	Stores history of alerts sent during booking updates or emergencies.	eventType, affectedReservation()
Reservation	Represents one booking, holds its l cycle state	reservationId, status, roomNumber, startTime, endTime, confirm(), cancel()

Class responsibilities:

Each class has its own responsibility. No one class is assigned to do everything. For example **Reservation** class has the responsibility of booking rooms. This work can not be done by the **Maintenance class** or any other class. One responsibility is completely specific to only one class.

TASK # 3 : Abstraction, encapsulation and invariant

- **Room**

Abstraction:

In real life a room has color, furniture, window position etc. But the Room class abstracts all of that and only keeps room capacity, room number, and accessibility status.

Everything else is abstracted.

Encapsulation:

In this Capacity and Accessibility Status are private. The Capacity or accessibility status can not be changed by any one except the admin.

Invariant:

Capacity of the room must stay positive it can not be negative number like -3.

$$\text{Capacity} > 0$$

- **Student**

Abstraction:

A real student has hobbies, types, personality, likes and dislikes and a lot more. This student class drops everything and keeps only student Id, enrolled courses.

Encapsulation:

Student Id

Accessibility Flag

Active reservations

Invariant:

The active reservations of the student can not be greater than 2. It needs to have a cap to exercise Fairness for all.

- **Reservation**

Abstraction:

Reservation in real time includes manual phone call, address, and booking details. But this specific class only includes reservation Id, start time and end time.

Encapsulation:

In this status() and confirm() are private and can only be viewed by the system security staff and admin.

Invariant:

Reservation can not hold a room that's already booked. That is why status can not be public.

Prevention of invalid states:

Encapsulation prevents invalid states by making sure an object's fields can only be changed through methods that verify the rules first. The fields can't be changed by direct access.

TASK # 4: Alternative designs and decision

Inheritance:

According to the concept of inheritance, in this model students extends person. Similarly facultyMembers extend person. Each person is one fixed subtype. If I use multi roles for the person, whenever I use the next role the previous one will get deleted from memory of system.

Composition:

Composition on the other hand can hold a list of roles for the class person. In my project person doesn't get turned into a student instead person holds a list of role objects, and a studentRole is one possible role.

COMPARISON

Multiple roles:

In inheritance the class person can not hold both students and faculty members. We will be forced to choose one subtype. While in case of composition we can easily add the second Role to the object list

Ease of changing permissions:

Inheritance requires creating a new object everytime someone's role changes in between semester. Composition just adds or removes a Role object from the existing Person.

Extensibility

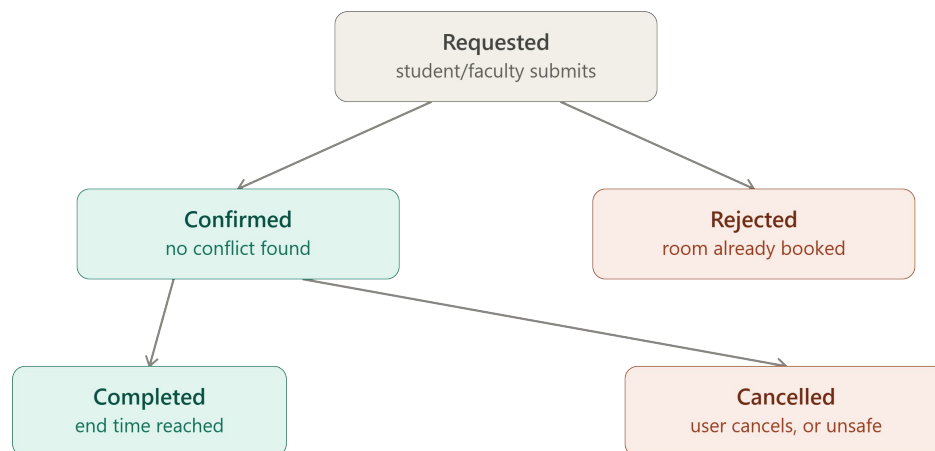
Inheritance require new subclass object every time new role is added while composition just add new role object to already existing user class.

DESIGN FOR THIS SCENARIO

As we can see that in our scenario, we need to change roles of a user in ongoing semester so the composition concept is better approach to use in our program because more or less it solves our problem of assigning new roles to previously existing user while keeping the history records.

TASK # 5 : Behavior and edge cases

- **Labelled state diagram**



Two simultaneous requests for the same space

When two persons try to book the same space at the same time reservation is the object that enforces it. The system checks the room status for reservation. First come First served rule is applied here. Whichever person comes first will get the space while the other person's request will be rejected because of the invariant, same room can not be booked by two persons at the same time.

Enforcing Object: Reservation

A maintenance block after confirmation

If maintenance staff finds that the room is unsafe after the room **reservation** is confirmed, the room reservation will be blocked **room.block()**. The reservation will be cancelled or relocated. The concerned bodies will get a notification (**Notification**) about the situation.

Enforcing Object:

- Reservation
- Room

An emergency cancellation or relocation:

If the whole N-Block gets closed for an emergency then the system will loop through all **reservations** and calls **relocate()** or **cancel()** on each one. Every change creates a **Notification**.

Enforcing Object:

- Reservation

TASK # 6: Independent exploration

Role Based Access Control (RBAC) is a security model in which permissions are assigned to roles, not directly to users. A user gets a permission by being assigned a role that possesses it, instead of the permission being given to them personally (Ferraiolo et al., 2001; National Institute of Standards and Technology, 2020). This makes access control easier to manage at scale, since it is the role that is managed by the administrators, not every users individual permission.

In my model, RBAC is precisely the mechanism used by the AccessRequest privacy rule. Instead of checking every users identity, the system checks the roles assigned to the user, and denies the request if the role is not SystemSupportStaff . The roles AdminStaff , Student , and SecurityStaff are explicitly denied for such an access, regardless of the users identity. It enforces the Task 1 privacy assumption and the Task 3 invariant by using roles, not personal checks, and it is compatible with the based-on-composition approach Role design I chose for the Task 4.

References

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security*, 4(3), 224–274. <https://doi.org/10.1145/501978.501980>

National Institute of Standards and Technology. (2020). *Security and privacy controls for information systems and organizations* (NIST Special Publication 800-53, Rev. 5). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>

TASK # 2: UML DIAGRAM

